

Análisis Individual — Caso C01

Sistema de citas de una clínica

Fundamentos de Ingeniería de Software · U1-E02 · Equipo Webtons (Proyecto XALTIL)
Martínez Aquino Jonnatan

1. Presentación del caso y consecuencias

El caso C01 describe una clínica pequeña que utiliza un sistema web para gestionar citas médicas. Aunque la aplicación permite registrar pacientes, médicos, fechas y horarios sin mostrar errores visibles, en la práctica surgen fallas que afectan a distintos actores. Los pacientes enfrentan la posibilidad de que su cita quede duplicada o se pierda sin explicación, lo que genera desconfianza en el servicio. El personal de la clínica, principalmente las recepcionistas, debe lidiar con conflictos de horario y con la carga adicional de corregir información manualmente cuando el sistema no refleja la realidad. A nivel organizacional, la clínica arriesga su reputación y su eficiencia operativa, ya que el tiempo que el personal dedica a resolver errores es tiempo que deja de dedicarse a la atención de pacientes. En conjunto, el caso muestra que un sistema puede funcionar en apariencia, pero seguir siendo poco confiable si no se acompaña de buenas prácticas de ingeniería de software.

2. Cinco problemas identificados

Problema 1: Reservas duplicadas por falta de control de concurrencia

Causa: el sistema no valida ni bloquea un horario en el momento en que dos personas intentan reservarlo casi simultáneamente.

Consecuencia: dos pacientes pueden quedar citados en el mismo horario con el mismo médico, generando conflictos y una mala experiencia.

Práctica de Ingeniería de Software: control de concurrencia mediante restricciones de unicidad en la base de datos y transacciones atómicas.

Evidencia del caso: “dos recepcionistas pueden reservar el mismo horario casi al mismo tiempo”.

Problema 2: Falta de trazabilidad en los cambios

Causa: el sistema no conserva un historial de quién modifica o cancela una cita.

Consecuencia: cuando surge un error, nadie puede identificar su origen ni asumir responsabilidad, lo que alimenta discusiones entre áreas.

Práctica de Ingeniería de Software: registro de auditoría (logging de cambios con usuario, fecha y acción realizada).

Evidencia del caso: “no existe un registro claro de quién modificó o canceló una cita”.

Problema 3: Canal de comunicación informal para los cambios

Causa: los cambios de cita se comunican por mensajería en lugar de actualizarse directamente en el sistema.

Consecuencia: la información del sistema queda desactualizada respecto a la realidad, generando citas fantasma o citas perdidas.

Práctica de Ingeniería de Software: establecer el sistema como única fuente de verdad y un flujo de actualización obligatorio ante cualquier cambio.

Evidencia del caso: “el personal suele avisar por mensajería y no siempre actualiza el sistema”.

Problema 4: Despliegues sin ambiente de pruebas

Causa: el equipo de desarrollo publica cambios directamente en producción cuando hay un problema urgente, sin pasar por un ambiente de prueba.

Consecuencia: se corre el riesgo de introducir nuevos errores mientras se intenta resolver uno existente, afectando a toda la clínica en tiempo real.

Práctica de Ingeniería de Software: uso de un ambiente de pruebas (staging) y control de versiones antes de publicar en producción.

Evidencia del caso: “publica cambios directamente en producción cuando la clínica reporta un problema urgente”.

Problema 5: Pruebas superficiales del sistema

Causa: las pruebas solo verifican que la pantalla abra y que sea posible guardar una cita, sin cubrir casos límite como reservas simultáneas.

Consecuencia: errores reales del sistema, como el de concurrencia, nunca se detectan antes de llegar a producción.

Práctica de Ingeniería de Software: diseño de pruebas funcionales y de casos límite (pruebas unitarias y de integración) que cubran escenarios reales de uso.

Evidencia del caso: “las pruebas se limitan a comprobar que la pantalla abra y que sea posible guardar una cita”.

3. Tres acciones prioritarias

Acción 1 (primer lugar): Implementar validación que impida reservar un horario ya ocupado

Justificación: es el problema que genera el daño más directo y visible a los pacientes; resolverlo evita gran parte de las discusiones derivadas de citas duplicadas.

Costo o limitación: requiere modificar la lógica de guardado y probablemente ajustar el diseño de la base de datos, lo que puede tomar más tiempo que un parche superficial.

Cómo comprobar la mejora: realizar pruebas de reservas simultáneas (dos usuarios intentando el mismo horario a la vez) y verificar que solo una se concrete.

Acción 2 (segundo lugar): Crear un registro de auditoría de cambios (quién, qué y cuándo)

Justificación: una vez resuelto el problema de duplicidad, la trazabilidad permite resolver disputas futuras y da confianza al personal sobre el origen de cada cambio.

Costo o limitación: implica agregar una tabla de auditoría y ajustar el código para registrar cada acción, lo que aumenta ligeramente la complejidad del sistema.

Cómo comprobar la mejora: verificar, durante un periodo de prueba, que cada modificación o cancelación quede correctamente registrada y sea consultable.

Acción 3 (tercer lugar): Definir un ambiente de pruebas separado del de producción

Justificación: evita que futuras correcciones urgentes introduzcan nuevos errores mientras se resuelve un problema existente.

Costo o limitación: requiere tiempo e infraestructura adicional, y puede ralentizar la respuesta ante emergencias en un inicio.

Cómo comprobar la mejora: medir el número de incidentes nuevos reportados después de cada despliegue, comparando antes y después de adoptar el protocolo.

4. Reflexión personal

Analizar este caso me hizo notar que, como futuro desarrollador, es fácil confundir que un sistema “funcione” con que un sistema sea confiable. Antes de este análisis, probablemente hubiera dado por bueno un sistema que permite guardar citas sin errores visibles, sin pensar en escenarios de uso simultáneo ni en la importancia de dejar rastro de cada cambio. A partir de este caso, me queda claro que debo dedicar más tiempo a diseñar pruebas que cubran situaciones reales, no solo el camino esperado, y a no tratar los despliegues urgentes como una excusa para saltarme buenas prácticas como el control de versiones o el uso de un ambiente de pruebas.

5. Fuentes consultadas y declaración de uso de IA

Para este análisis se utilizó únicamente el material proporcionado por la docente (documento de casos U1-E02) como fuente principal, sin consultar fuentes externas adicionales.

Uso de IA: Utilicé Claude (Anthropic) para organizar la estructura del análisis y redactar un primer borrador a partir de mis propias ideas sobre el caso. Verifiqué el resultado comparándolo directamente con el texto del caso proporcionado por la docente y ajusté la redacción final para que reflejara mi propio criterio.